

32. News Aggregator (Google News / Feedly-style)

Interviewer: Design a news aggregator — think Google News or Feedly. We pull articles from thousands of external sources (RSS feeds, publisher APIs, some crawling), and users open the app to read a feed of stories, browse by category, and follow topics. When the same story is covered by 200 different outlets, we should show it as **one story with multiple sources**, not 200 duplicate entries. Design the backend.

Candidate: This is a different animal from a web crawler or a social news feed. We don't control the writers — we're **consuming** a firehose of independently-published content, and the hard part isn't "store and rank posts," it's **noticing that fifty newsrooms just wrote about the same event** and turning that into one clean, ranked, personalized story. Let me clarify scope first.

1. Clarifying the requirements

Candidate: A few questions: 1. How do articles get to us — RSS polling, publisher push APIs, or do we also crawl pages ourselves when a source has no feed? 2. When outlets cover the same event, do we need to **merge them into one story** (Google News does this — "482 outlets covering this"), or is per-article display fine? 3. Personalization — ranked-for-you feed, or just reverse-chron / editorially-curated categories? 4. How fresh does "fresh" need to be — seconds for breaking news, or is a minute or two of lag acceptable?

Interviewer: Mostly RSS polling plus a handful of publisher push APIs; assume light crawling as a fallback for feed-less sources, but don't design a crawler — that's a solved upstream problem, focus on what happens *after* we have an article. Yes — same-story deduplication into a single **story cluster** is the core requirement, not optional. Support both a personalized ranked feed and category feeds (Top Stories, Sports, Tech, Local). A couple of minutes of lag is fine for normal stories; **breaking news should surface within tens of seconds.**

Candidate: Good, that's a clear shape. Requirements:

Functional - Ingest articles continuously from ~50,000+ sources (RSS/API/light crawl). - **Deduplicate** near-identical articles (wire-service syndication, copy-paste rewrites). - **Cluster** distinct-but-related articles about the same real-world event into one **story**. - Rank stories into a personalized home feed and fixed category feeds. - GET /feed , GET /category/{name} , GET /story/{id} (article + all covering sources).

Non-functional — this is where it's interesting - **Read-heavy, by a wide margin:** millions of feed reads per second against a comparatively tiny stream of new articles. Classic case for precomputing and caching. - **Ingestion is bursty and heterogeneous:** some sources publish every 30 seconds (wire services), most a few times a day. One fixed poll rate wastes capacity and misses breaking news. - **Dedup/clustering must not become $O(n^2)$:** a breaking story brings hundreds of near-identical articles within minutes; comparing every new one against everything published today doesn't scale. - **Freshness**

vs. cost: a story should appear in feeds quickly, but re-ranking/re-clustering the whole corpus per request is wasteful — most of that work is shared across every reader of the same category.

2. Estimation — the two numbers that define the design

Sources: ~50,000 feeds/publishers. ~500 "high-frequency" (wire services, majors) polled every 30–60s; ~49,500 long-tail sources polled every 5–15 min.
poll requests/sec $\approx (500/45s) + (49,500/600s) \approx \sim 95$ fetches/sec avg
(bursty; a scheduler smooths this, see §4)

Articles ingested: ~5M/day globally ≈ 58 /sec avg, bursts to 500–1,000/sec during breaking news (many outlets filing within minutes).

Distinct STORIES after dedup+clustering: the same event is covered by 20–500 outlets; dedup ratio ~10:1 $\rightarrow$ ~500K distinct stories/day. This is the number that matters for ranking/serving — 10x smaller than raw volume.

Readers: 100M DAU, ~5 feed opens/day $\rightarrow$ 500M feed reads/day
 $\approx 5,800$ reads/sec avg, peak $\approx 30,000$ reads/sec (commute hours, breaking news).
Read:write ratio on the SERVING side $\approx 30:1$ — reads dominate.

Storage: 5M articles/day * ~5KB ≈ 25 GB/day text; embeddings (768-dim float32 ≈ 3 KB/article) ≈ 15 GB/day of vectors. ~1.8B articles/year, ~10TB text, ~5TB vectors — NOT a storage-volume problem, it's a "compare-cheaply" one.

$\Rightarrow$ Two numbers define this design: (1) ~95 fetches/sec spread across 50,000 sources at wildly different natural rates — an adaptive polling problem, not a raw-QPS one; and (2) clustering 500 near-duplicate articles/minute during a breaking story without comparing each one to the whole corpus.

Candidate: So unlike the burger giveaway (one hot counter) or the news feed (fan-out on follower count), this problem's core tension is **"how do we tell 500 near-identical documents apart from each other, cheaply, in near-real-time, and then serve the ranked result to millions of readers without redoing that work per request."**

3. A simple V1 (and where it breaks)

Candidate: Simplest possible thing: poll every source every 60 seconds, store each article as its own row, and when a user opens the feed, run a SQL query for recent articles ordered by publish time.

```
[ Poller: loop over all 50,000 sources every 60s ] --> [ Postgres: articles table ]
|
[ User opens feed ] --> SELECT * FROM articles
                        WHERE category = ? ORDER BY published_at DESC LIMIT 50
```

Why it dies: - **No dedup.** A user sees the same headline 40 times from 40 outlets — unusable product. - **Fixed 60s polling wastes capacity on dead sources** and is too slow for wire services that publish every 30s, while hammering tiny blogs that post twice a week. - **Ranking at read time re-scans the**

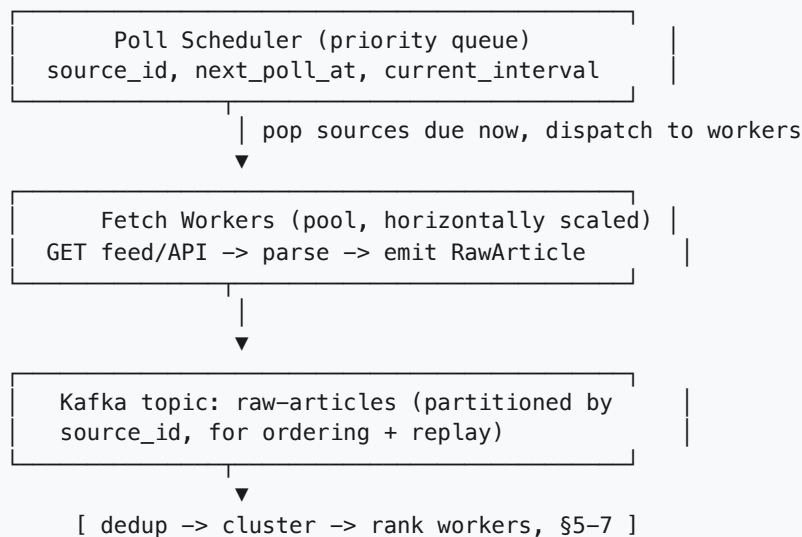
table on every one of 30,000 reads/sec. Even with an index, computing freshness + popularity + personalization per request, per user, against a growing table is the same "recompute shared work per request" mistake as scanning a hot row — just spread across reads instead of writes. This is exactly what Deep-dive A digs into.

The real system needs four things the naive version doesn't: **adaptive ingestion**, **dedup**, **clustering**, and **precomputed, cached ranking** — each solving a piece of "500 outlets, one story."

4. Scaling ingestion — polling thousands of sources at different rates

Interviewer: You can't poll 50,000 feeds on one timer. How do you actually schedule this?

Candidate: Treat polling as a **priority queue keyed by next-poll-time**, not a fixed loop.



- **Adaptive interval per source:** track how often a source actually publishes new content. A source that drops a new article every ~40s gets polled every ~30s; a blog that posts twice a week gets polled every few hours. Back off on empty polls, tighten up when a source turns unusually active (breaking news). Same idea as HTTP caching — use `ETag / Last-Modified` so most polls cost a cheap 304, not a full re-fetch.
- **Push where available.** Some publishers offer webhooks/PubSubHubbub — prefer push over poll whenever it exists; it eliminates polling delay and wasted requests entirely for those sources.
- **Kafka as the ingestion buffer**, not direct writes to storage. Fetch workers just produce `RawArticle` events; this decouples fetch rate from dedup/cluster/index throughput, and gives replay if a downstream stage falls behind or crashes.
- **Why not one shared poll loop over all sources sequentially?** A single slow/hanging source would stall the whole loop. Independent workers pulling from a shared priority queue mean one bad source degrades only its own slot.

5. Dedup — catching near-identical articles at ingestion

Interviewer: A wire-service story gets picked up verbatim (or lightly reworded) by 200 outlets. How do you recognize that at ingestion time, before it clutters everything downstream?

Candidate: Two layers, cheapest check first, because most articles are NOT duplicates and we don't want every one to pay for an expensive check.

New article arrives

- ① Exact/near-exact check — SimHash / MinHash over shingles
 - shingle the article body into overlapping k-word tokens
 - hash into a compact fingerprint (SimHash: 64-bit; MinHash: N hash functions)
 - compare Hamming distance (SimHash) or Jaccard estimate (MinHash) against RECENT fingerprints (last ~72h) via LSH buckets — NOT a full scan (§13)
 - catches: wire-service syndication, copy-paste rewrites, same press release
 - close match found → mark as duplicate/near-duplicate, attach to same article
 - no close match ↓
- ② Semantic check — embedding similarity
 - embed the article (title + lede) into a dense vector
 - cosine-similarity search in a vector index against recent story embeddings
 - catches: same event, DIFFERENT wording — one outlet's original reporting, another's independent paraphrase — things SimHash/MinHash miss entirely
 - high similarity to an existing STORY → candidate for clustering (§6)
 - low similarity everywhere → new distinct article/story

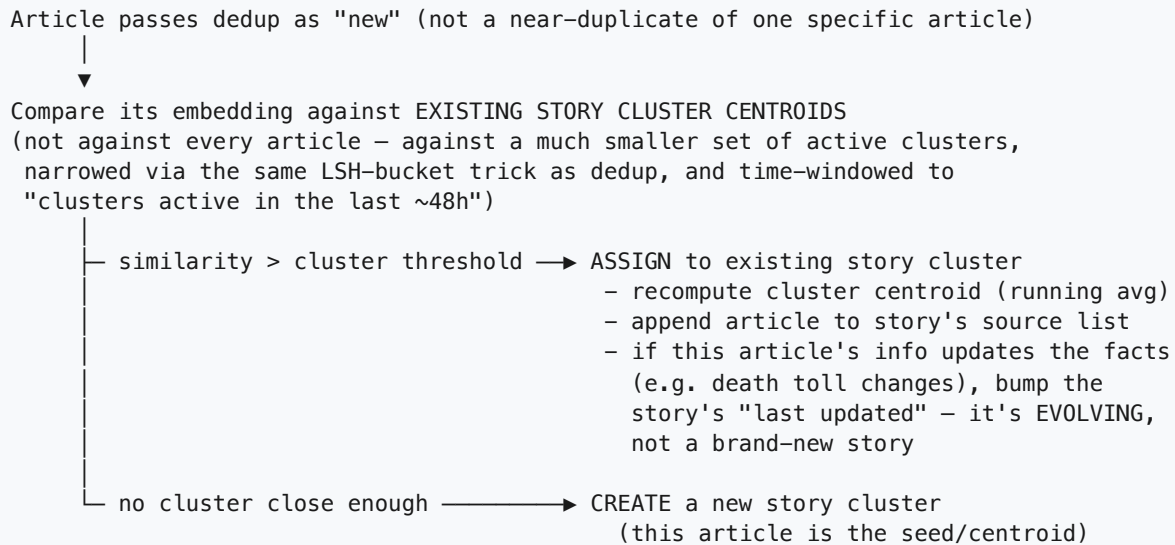
- **Why not just hash the raw text and compare for exact equality?** Real syndication almost always has *some* difference — a different headline, an editor's note, ad markup. Exact hashing misses all of that; near-duplicate detection (shingling + MinHash/SimHash) tolerates small textual differences while staying cheap. **MinHash + LSH banding** is what lets us avoid comparing every new article to every old one: articles bucket by hash bands, and we only compare within the same bucket — turning an $O(n^2)$ all-pairs problem into near- $O(n)$ bucket lookups. This is the mechanism we lean on hardest in Deep-dive B.
- **Why not skip straight to embeddings for everything?** Embedding + nearest-neighbor search is more expensive (compute + vector index lookup) than a fingerprint comparison, and pure copy-paste duplicates are extremely common (wire syndication). Cheap fingerprinting filters the easy 80% of duplicates before anything touches the more expensive embedding path.

6. Story clustering — grouping related coverage into one story

Interviewer: Dedup catches copies of the *same text*. What about 40 outlets who each wrote their *own* article about the same event — different words, same story? That's the actual "Google News" feature.

Candidate: That's clustering, one layer up from dedup: dedup says "these two documents are basically the same text"; clustering says "these N documents, though textually different, describe the same real-

world event, and should render as one story with N sources."



- **Incremental, streaming clustering – not batch re-clustering.** We assign each new article to a cluster (or start a new one) as it arrives, rather than periodically re-running k-means over the whole corpus. Streaming assignment is what lets a story appear within seconds of the first few articles, and scales with new-article rate, not corpus size.
- **Why not just dedup and skip clustering – one article "wins," the rest hidden as duplicates?** The outlets are NOT duplicates of each other's text – they're independent reporting on the same event, often with different facts, quotes, angles. Hiding 39 of them loses information users want ("compare coverage"); clustering keeps them all attached to one story card, the actual product requirement.
- **Cluster centroid drift for evolving stories.** A story that starts as "small plane crash reported" and evolves over 6 hours into "airline confirms cause, 12 dead" is still one story – new articles attach to it, and the *headline/summary shown* tracks the latest, most-corroborated version. We keep a `canonical_article_id` per cluster, reassigned as better/more-authoritative coverage arrives (Deep-dive B has the details).

7. Ranking, personalization, and category feeds

Interviewer: Now I have story clusters instead of raw articles. How do stories get ordered in a user's feed?

Candidate: A blended score, computed **per story, not per user**, plus a lightweight personalization pass on top:

```

base_score(story) = w1 * freshness(story)      -- exponential decay since first-seen
                  + w2 * popularity(story)     -- click/read velocity, # covering outlets
                  + w3 * source_authority(story) -- avg trust score of contributing outlets

-- computed ONCE per story, shared by every reader (this is the expensive part)
-- we do NOT want to repeat per request – see Deep-dive A)

personalized_score(user, story) = base_score(story)
                                + w4 * topic_affinity(user, story.topics)
                                + w5 * source_affinity(user, story.sources)
                                - penalty(already_seen(user, story))

-- this part is cheap: a dot-product against a user's interest vector,
-- applied on top of an already-ranked, already-small candidate list

```

- **Category feeds (Top Stories, Sports, Tech, Local)** are just `base_score` ranked within a topic tag — fully shared across all users, recomputed on a short cycle (seconds, driven by new-story arrival), cached once for everyone reading that category. Massively cheaper than personalizing per user, since most of the internet reads the same "Top Stories."
- **The personalized home feed** takes the small, pre-ranked candidate pool per category/topic and re-scores it against each user's interest profile at serve time — cheap precisely because the expensive shared work (freshness + popularity + clustering) already happened, off the read path.
- **Freshness decay** matters most for breaking news (a 20-minute-old plane-crash story should outrank a 2-hour-old one with more clicks, initially) but decays in favor of popularity as a story ages and its "still developing" signal fades.

8. API + data model

```

GET /feed?cursor=...      -> personalized ranked stories
GET /category/{name}?cursor=... -> category-ranked stories (shared across users)
GET /story/{storyId}      -> canonical article + full list of covering sources
POST /interactions {userId, storyId, action} -> click/read signal (async, feeds ranking)

```

Data model — why a mix of stores, not one:

Data	Store	Why
Raw article text + metadata	Document store (DynamoDB/Cassandra)	High write volume, simple <code>article_id</code> lookups, no joins needed; scales horizontally with ingestion.
Story clusters (sources, centroid, canonical article)	Document store , keyed by <code>story_id</code>	A growing source list fits a document naturally; relational would need a join table on every read.
Similarity search (dedup + clustering)	Vector index (ANN service)	Purpose-built nearest-neighbor lookups at this volume/latency; a SQL <code>WHERE</code> clause can't do approximate nearest-neighbor search.
Full-text search	Elasticsearch/OpenSearch	Inverted-index text search + faceting by source/date/category — not something a document store does well.

Data	Store	Why
Precomputed ranked feeds	Cache (Redis)	Sub-100ms reads at 30,000+/sec; recomputing rank per request is the anti-pattern Deep-dive A avoids.
User interest profiles	Key-value store (Redis/DynamoDB)	Small, hot, point-lookup on every personalized request — no transactions needed.

Why not one relational database? Nothing needs cross-entity ACID (an article insert doesn't need to be atomic with a cache update); the shapes are too different — append-only blobs, growing lists, high-dimensional vectors, hot KV profiles — forcing them into SQL tables would mean denormalizing anyway or paying for joins nothing here needs.

9. Serving path — precomputed feeds, caching, multi-region

Interviewer: Walk me through what actually happens when 30,000 users/sec open the app.

Candidate: Almost none of that traffic should reach the ranking/clustering pipeline — it should hit a cache of feeds that were computed **once**, shortly after ingestion, and reused by everyone.

```

[ Dedup -> Cluster -> Rank pipeline ($5-7) ] (near-real-time, seconds-scale lag)
      |
      | writes ranked story lists per category/topic
      v
[ Redis: precomputed feed cache ]
  category:top-stories -> [story_17, story_42, story_9, ...] (TTL ~10-30s)
  category:sports      -> [...]
  user:{id}:candidate_pool -> small pre-ranked pool per user's followed topics
      |
      v
[ CDN edge cache ] -- category feeds are IDENTICAL for millions of users;
                   cache at the edge, short TTL, near-zero origin load
      |
      v
[ 30,000 reads/sec from users ] -- almost all served from CDN/Redis, not
                                the ranking pipeline

```

- **Category feeds cache extremely well** — a million users reading "Top Stories" read the exact same list, so CDN + Redis absorb nearly all that traffic.
- **Personalized feeds** can't be edge-cached per-user, but the expensive part (clustering + base scoring) is still shared; only the final cheap re-score against a user's interest vector happens per-request, against a small candidate pool — not the full corpus.
- **Multi-region:** ingestion (polling + dedup + clustering) runs centrally, since a story is a story regardless of reader location. The **read path** replicates the small "current feed" dataset (a few thousand ranked story-IDs per category) to every region — trivially small to replicate widely, so readers everywhere get local, low-latency reads even though clustering happens centrally.

10. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Ingestion buffer	Kafka (partitioned by source_id)	Durable, replayable, absorbs bursty fetch spikes and decouples fetch rate from downstream dedup/cluster throughput. <i>Not direct synchronous writes to storage</i> — a slow downstream stage would then throttle fetch workers and delay ingestion of unrelated sources.
Raw article store	Document store (DynamoDB/Cassandra)	Simple article_id lookups at high write volume; horizontally scales with source count. <i>Not Postgres</i> — no relational structure is needed for what's essentially write-once, read-by-key blobs at this volume.
Similarity/embedding search	Vector index (ANN service)	Purpose-built approximate nearest-neighbor search at sub-second latency over millions of vectors; the whole dedup/clustering design depends on NOT doing brute-force comparisons (§13). <i>Not a SQL similarity UDF</i> — a per-row distance computation over millions of rows per new article is the $O(n^2)$ trap we're explicitly avoiding.
Near-dup fingerprinting	MinHash/SimHash + LSH banding	Turns "compare to everything" into "compare within your hash bucket," catching copy/paste and lightly-reworded syndication cheaply. <i>Not embeddings for this layer</i> — fingerprinting is far cheaper and catches the highest-volume duplicate case (verbatim wire copy) without needing a model inference call per article.
Full-text search	Elasticsearch/OpenSearch	Relevance-ranked text search + faceting (by source, date, category) for the search feature; inverted indexes are the right data structure, not something a KV or document store provides.
Feed cache	Redis	Sub-ms reads for precomputed category/candidate-pool feeds at 30,000+ reads/sec; short TTLs let freshness flow through without recomputing per request. <i>Not recomputing rank per request</i> — that repeats shared, expensive work for every one of millions of readers (Deep-dive A).
Edge delivery	CDN	Category feeds are identical across huge swaths of users; caching at the edge removes nearly all read traffic from the origin entirely.
Interaction signals (clicks/reads)	Kafka -> async aggregation workers	Click/read events feed ranking (popularity) and personalization, but must never sit on the read critical path; aggregated asynchronously into rolling counters.

The one-sentence justification you'd say out loud: "MinHash/LSH and embedding similarity turn 'is this a duplicate of anything' from an $O(n^2)$ comparison into a bucketed lookup; that lets us cluster stories incrementally in near-real-time instead of batch re-clustering; and because clustering and ranking are computed once per story and cached — in Redis and at the CDN edge — millions of feed reads per second never touch that expensive pipeline at all."

11. Consistency model — what's strong, what's eventual

- **Strong-ish** (no "user's own data" to read back here, so read-your-writes doesn't really apply): the dedup fingerprint index needs to reflect an article the moment it's ingested, or a duplicate published seconds later could slip past as "new." We accept a small window where duplicates could briefly double-count.
 - **Eventual, and deliberately so:** cluster assignment, ranking scores, and the feed cache. A story's cluster might briefly show 38 sources instead of 41 because the last three haven't finished processing; a feed might be 10-30 seconds stale (the cache TTL). Both are the right trade — nobody notices a missing 41st source for a few seconds, and it's far cheaper than recomputing on every read.
 - **What must not drift silently: canonical article/headline for an evolving story.** If breaking-news facts change (death toll, suspect named), the *displayed* summary must update promptly even though cluster membership is eventually consistent — handled by bumping a `version / last_updated` field that busts that story's cache entry immediately, independent of the broader feed TTL.
-

12. Failure modes & graceful degradation

Failure	What happens	Mitigation
A source's feed goes down/malformed	That source stops contributing articles	Poller backs off automatically; other 49,999 sources unaffected — no shared state per source
Dedup/cluster pipeline lags behind ingestion	Articles pile up unclustered in Kafka	Kafka buffers; consumers catch up; readers see the last-good cached feed, slightly stale but never wrong
Vector index (ANN service) unavailable	Can't check semantic similarity	Fall back to fingerprint-only dedup, create clusters more liberally; reconciliation job merges over-fragmented clusters once recovered
Redis feed cache cold/evicted	Feed reads would miss cache	Serve a slightly-stale secondary tier, or a "safe default" (global Top Stories) rather than falling through to the full pipeline under load
Breaking-news burst overwhelms clustering	Same story briefly forms several clusters	Acceptable short-term; the merge pass (Deep-dive B) reconciles within minutes
One source is malicious/spammy	Floods ingestion with junk	Per-source rate caps at ingestion; trust score down-weights it in ranking without blocking ingestion

13. Follow-up curveballs

Interviewer: How do you stop the same story from bouncing between two clusters — one gets created, then five minutes later a near-identical second cluster forms from a slightly different angle on the same event?

Candidate: A periodic **cluster-merge pass**: compare centroids of recently-created clusters against each other — a much smaller set than all articles — and merge any pair above a high similarity threshold, keeping the earlier `story_id` as canonical and unioning source lists. Cheap because it's cluster-to-cluster, not article-to-article.

Interviewer: A user wants to follow a specific topic, not just categories — say "electric vehicles." How does that fit the ranking design?

Candidate: Topics are tags attached to a story cluster at ingestion (same embedding/ classification step used for categories, just finer-grained). "Follow a topic" is functionally identical to a category feed — a shared, precomputed, cached ranked list — just with a longer tail of smaller audiences; below some size threshold we compute it lazily instead of continuously.

Interviewer: How do you handle a correction or retraction on a published story?

Candidate: Corrections update the specific article/story document (bump `version`, invalidate that story's cache entry — same mechanism as evolving breaking news in §11) rather than touching the broader feed cache. If severe (full retraction), flag it and exclude it from the next scoring pass — no need to instantly purge already-rendered feeds.

Interviewer: Could a bad actor game ranking by spinning up 50 fake "outlets" that all republish one story to inflate its "covered by 50 sources" signal?

Candidate: Yes — that's why `source_authority` is a *trust score* per source, not a raw count; unverified sources contribute near-zero popularity weight until they build a track record. The near-duplicate fingerprint check (§5) would also likely catch identical republished text, collapsing those 50 "sources" into one low-trust duplicate.

14. Deep-dive: "Why not re-rank and re-cluster on every user request?"

Interviewer: All this precomputing and caching feels like a lot of machinery. Why not keep it simple — when a user opens the feed, query recent articles, cluster them, and rank them right there, live? Then it's always perfectly fresh and there's no cache-invalidation complexity to think about.

Candidate: It's tempting because it sounds simpler, but it fails for the same underlying reason a hot database row fails under 670k writes/sec in the burger problem — **it repeats expensive, shared work for every single request instead of once for everyone who'd get the same answer.**

What "live" actually costs, per request

Per-request live pipeline for ONE feed open:

1. Pull candidate articles from the last N hours (thousands of rows)
2. For each, run/re-run near-dup + embedding similarity checks against each other to (re-)build clusters <- the expensive part
3. Score every cluster (freshness + popularity + authority)
4. Personalize against this user's interest vector
5. Return top 50

Step 2 alone, done fresh, is exactly the $O(n^2)$ -shaped comparison problem (§13) — and CATEGORY FEEDS ARE IDENTICAL ACROSS MILLIONS OF USERS. Doing step 1–3 fresh for every one of 30,000 reads/sec means redoing the SAME clustering/ranking computation 30,000 times a second, when the answer for "Top Stories right now" is the same for nearly everyone asking.

Comparison

Concern	Cluster + rank live, per request	Precompute near-real-time, serve from cache
Latency at 30,000 reads/sec	Clustering/ranking (seconds-scale) sits in the request path — feed opens take seconds, not ms	Reads hit Redis/CDN in low single-digit ms; expensive work already happened, off the read path
Cost	Redoes identical work for millions of near-identical requests (same category, same window)	Shared work paid once per refresh cycle; personalization on top is a cheap dot product
Freshness	"Live," but degrades under load exactly when it matters (breaking news = traffic spike = slowest computation)	Near-real-time lag via streaming clustering; holds up under load because serving is decoupled from computing
Failure blast radius	A slow clustering step directly stalls user-facing reads	A lagging pipeline just serves a slightly-stale cached feed — reads stay fast
Complexity	No cache invalidation, but the clustering pipeline must hit <i>interactive</i> latency at <i>peak concurrency</i> — a harder bar	Cache TTLs / versioned invalidation for specific stories — real but bounded complexity

Candidate: The tell is that this is a **read:write ratio of roughly 30:1 on top of work that is shared, not personalized, for most of the feed.** Whenever many readers would get the identical answer, computing it once and caching it dominates computing it per-reader — this is the same principle as the flash sale's CDN-cached product page, just applied to "the top stories right now" instead of "is this item in stock." Live computation is the right instinct for a single user's *personalization delta*, which is why we still do that part per-request — just on top of a small, already-ranked pool, not the whole corpus.

Rule of thumb: *If the answer to a request is identical (or nearly identical) for a large fraction of requesters, compute it once and cache it — don't let the read path pay for work that's shared.*

Reserve per-request computation for the part that's genuinely request-specific (here: personalization against one user's interest vector).

15. Deep-dive: 500 outlets, one breaking story, in minutes

Interviewer: Let's stress the hardest case directly. A major event breaks. Within 10 minutes, 500 different outlets publish articles about it — some verbatim wire copy, some independently written, some updated twice as facts change. Walk me through exactly how you detect the near-duplicates and cluster them **without comparing every new article to every other article**, and how you handle the story *evolving* under you the whole time.

Candidate: This is the scenario the whole dedup/clustering design (§5-6) is built around. Let me walk the timeline.

The naive failure mode first

If every new article is compared against EVERY existing article/cluster today:
article #1 of the burst: compare against ~500K stories already published today
article #500 of the burst: compare against ~500K + 499 others
500 articles * ~500,000 comparisons each ≈ 250,000,000 comparisons
in a 10-minute window → the comparison step alone becomes the bottleneck,
arriving exactly when speed matters most (breaking news).

That's the $O(n^2)$ -shaped trap: comparison cost grows with the *entire* corpus, not with the burst size, and it gets worse the longer the day/system has been running.

The timeline, done right

t=0:00 Outlet A publishes original reporting. Embedding + fingerprint computed. No close match in any LSH bucket -> NEW cluster (story_9001, 1 source).

t=0:01 Outlet B's near-verbatim wire copy lands in the SAME LSH bucket as A's fingerprint -> matched in O(1) bucket lookups, not a corpus scan. Marked duplicate, attached to story_9001. (2 sources.)

t=0:03 Outlets C-Z (23 outlets) publish independently-written coverage - different wording, same event. Fingerprints don't match -> fall through to embedding search against RECENT/ACTIVE cluster centroids only (bucketed, time-windowed to 48h) -> matches story_9001's centroid -> ASSIGNED, never compared to the other 499,999 stories today. (25 sources.)

t=0:07 400 more outlets file over four minutes. Same bucketed path scales because cost per article stays roughly constant regardless of corpus size. (story_9001: 425 sources.)

t=0:12 Outlet Z updates its article - death toll revised, cause named. Same story_9001 EVOLVING, not a new story: re-score triggers, and if Z's update is now most-corroborated + most-recent + highest-authority, canonical_article_id reassigns to it -> displayed headline updates, no second cluster created.

t=0:15 A smaller story_9034 (6 sources) formed from a slightly different embedding angle before the corpus had enough signal. The periodic cluster-MERGE pass compares recent centroids (small N, cheap) and merges story_9034 into story_9001.

Near-duplicate techniques, ranked by what they catch

1. **MinHash/SimHash + LSH banding (fingerprint layer).** Catches verbatim/near-verbatim syndication cheaply, in O(1)-ish bucket lookups instead of scanning — the highest-leverage, lowest-cost defense, since much of a breaking-news burst genuinely is wire-service copy.
2. **Embedding similarity against a narrowed candidate set (semantic layer).** Catches independently-written coverage that fingerprinting misses. The key design choice: compare against **recent, active cluster centroids only**, never the full historical corpus, keeping the candidate set small regardless of corpus size.
3. **Time-windowing.** Only clusters active in the last ~48h are candidates for new assignment — a months-old story about an unrelated topic should never be a match, keeping the active-cluster set bounded even as historical stories grow into the millions.
4. **Periodic cluster-merge pass, not per-article global comparison.** Short-term fragmentation (two clusters, one event) is acceptable; a cheap periodic cluster-to-cluster merge (thousands of centroids, not millions of articles) cleans it up within minutes, trading brief fragmentation for avoiding an all-pairs comparison entirely.

Handling story evolution specifically

The key insight: an evolving story is not a clustering problem, it's a **canonical-pointer problem** on top of clustering that already succeeded. Once articles are grouped into story_9001, "what headline do we show" is a small, fast decision — re-score the cluster's articles by recency + authority + corroboration

whenever a new one arrives, and reassign `canonical_article_id` if a newer one wins. Cheap regardless of how many outlets are in the cluster.

What to say out loud: *"The trick that prevents $O(n^2)$ is that no new article ever compares against the whole corpus — fingerprint hashing buckets it against near-identical copies in $O(1)$, and embedding search narrows candidates to recently-active cluster centroids, not every story ever published. A little short-term fragmentation — two clusters for one event — is fine, because a cheap periodic cluster-to-cluster merge pass cleans it up in minutes without ever doing an all-pairs comparison. And an evolving story is just a canonical-article reassignment inside an already-correct cluster, not a re-clustering problem."*

Summary — the mental model

A news aggregator is **not a storage or a fan-out problem — it's a "recognize that 500 independently-published documents describe one event, cheaply and fast, then serve the ranked result to millions of readers without redoing that recognition per reader" problem**. The winning moves: **adaptive per-source polling** so 50,000 sources at wildly different publish rates don't waste capacity or miss breaking news; **fingerprint-based near-dup detection (MinHash/SimHash + LSH)** to catch syndicated copies in $O(1)$ -ish bucket lookups; **embedding similarity against a narrow, time-windowed set of active cluster centroids** — never the whole corpus — to catch independently-written coverage of the same event; **incremental streaming clustering** with a cheap periodic merge pass to reconcile short-term fragmentation; and **precomputed, cached ranking** (Redis + CDN) so the expensive shared work of clustering and scoring happens once per story, not once per one of millions of feed reads. Freshness comes from keeping the pipeline fast and near-real-time, not from computing on demand — the read path stays fast precisely because it never repeats work the write path already did for everyone.